🐳 Final Project - Containerization Technologies

Build a microservices application with Docker

Information	Details
📅 Year	2025-2026
👥 Team	3-4 students
⏱️ Presentation	9-10 min + 5 min Q&A

🎯 Mission

Develop a web application based on **microservices architecture** that solves a real problem.

🤔 Why This Project?

This final project simulates a **real-world professional environment**. In companies today:

Reality	What You'll Learn
Applications are containerized	Build production-ready Docker images
Teams work on different services	Collaborate on a multi-service architecture
Security is non-negotiable	Implement security scanning in CI/CD
Deployments are automated	Create pipelines that build and deploy
Systems must be resilient	Design isolated, persistent services

> 💼 ***Career Impact:*** *Container skills are among the most requested in DevOps, Backend, and Cloud job postings.*

👥 Why Teamwork Matters

This project is meant to done with teamwork and that's intentional.

Skill	How You'll Practice It
Communication	Daily syncs, code reviews, merge requests
Task Division	Each member owns a service or component
Conflict Resolution	Handling merge conflicts, design disagreements
Documentation	Writing for teammates, not just yourself
Accountability	Your team depends on your deliverables

> 🎓 ***What employers look for:*** *Technical skills get you the interview. Collaboration skills get you the job.*

🏆 What Makes a Great Project?

Level	Characteristics
⭐ Basic	It works. Minimum requirements met.
⭐⭐ Good	Clean code, good structure, proper documentation
⭐⭐⭐ Excellent	Creative idea, bonus features, polished presentation
🏆 Outstanding	Production-ready quality, impressive demo, great teamwork visible

---

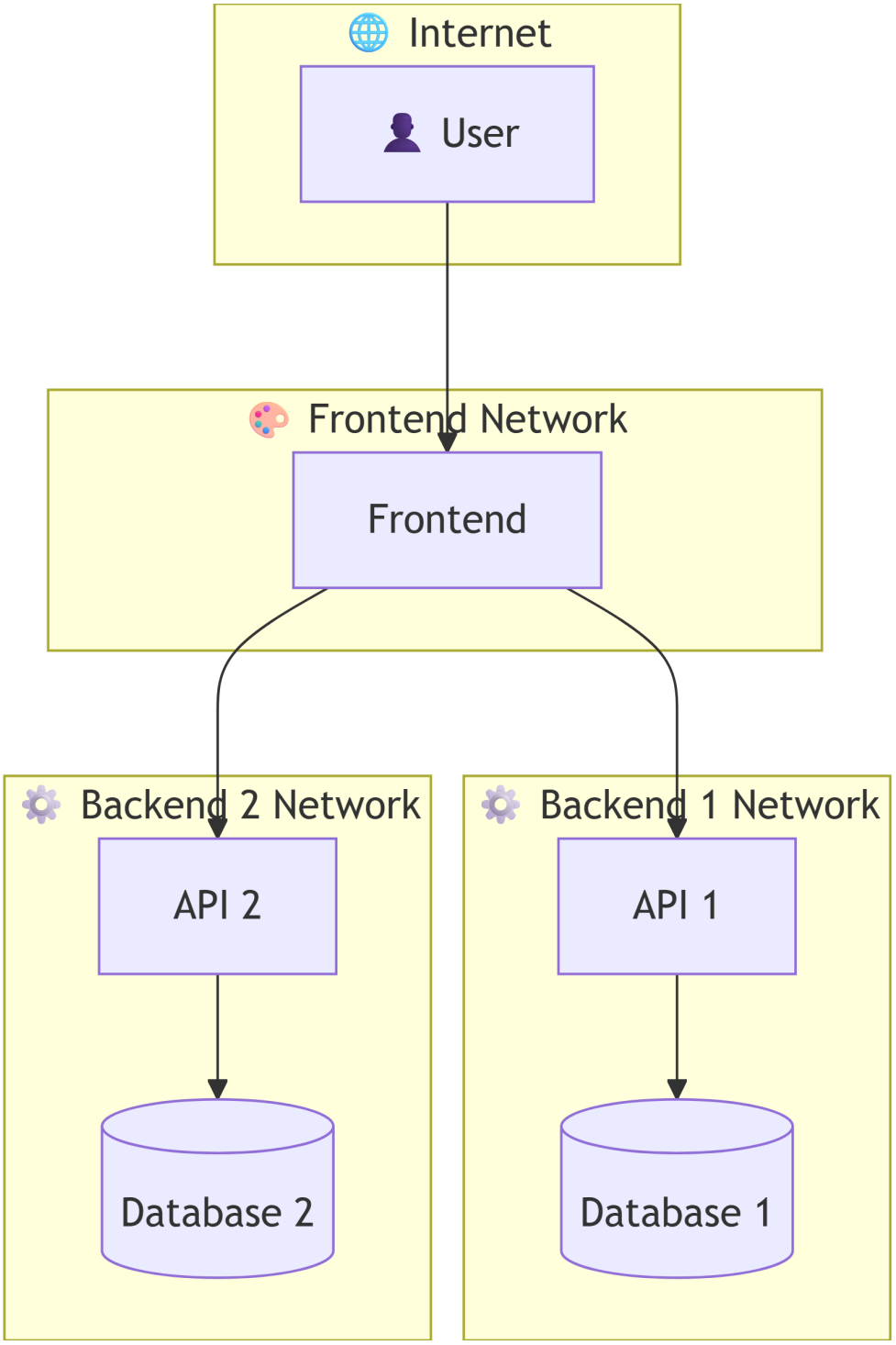
📦 What You'll Deliver

Your application will demonstrate best practices at all levels:

- ✅ Container orchestration with Docker Compose
- ✅ Network isolation between services
- ✅ CI/CD pipeline with security scanning
- ✅ Deployment to Docker Hub
- ✅ Clean, documented codebase
- ✅ Professional presentation

🏗️ Required Architecture

Your application must have this minimum structure:



Network Isolation Rules

✅ Allowed	❌ Forbidden
Frontend → API 1	API 1 → Database 2
Frontend → API 2	API 2 → Database 1
API 1 → Database 1	Host → Any Database
API 2 → Database 2	Database 1 ↔ Database 2

> ⚠️ *You must prove network isolation during your presentation.*

📋 Technical Requirements

1️⃣ Application Components

Component	Requirement
Frontend	1 service with **hot reload** (bind mount)
Backend APIs	Minimum **2 microservices**
Databases	**1 per backend**, isolated

2️⃣ Docker Images

Requirement	Details
Base images	**Pinned versions** (no `:latest`)
Image size	As small as possible (alpine/slim preferred)
Security	`USER` instruction (non-root)
Health	`HEALTHCHECK` instruction

3️⃣ Data Persistence

Requirement	Details
Volumes	One **named volume** per database
Naming	Logical names (e.g., `api1-db-data`, `api2-db-data`)
Survival	Data must persist after `docker compose down`

4️⃣ Hot Reload (Frontend)

Requirement	Details
Method	**Bind mount** of source code
Behavior	Changes in code → instant update in browser after refresh
No restart	Container must NOT restart to see changes

5️⃣ CI/CD Pipeline

Your pipeline must run on **Gitea Actions** (git.zohrabi.cloud).

Stage	Tool	Requirement
Security Scan	`aquasec/trivy`	Fail on MEDIUM/HIGH/CRITICAL
Build	`docker/build-push-action`	All service images
Push	Docker Hub	Public images

6️⃣ Git Management

Requirement	Details
Platform	**git.zohrabi.cloud** (Gitea)
Branches	Feature branches + merge requests
Commits	Clear, descriptive messages
Collaboration	All team members contribute

📁 Repository Structure

```
.
├── .gitea/
│   └── workflows/
│       └── ci.yaml              # CI/CD pipeline
├── frontend/
│   ├── Dockerfile
│   └── src/                     # Source code (bind mounted)
├── api1/
│   ├── Dockerfile
│   └── src/
├── api2/
│   ├── Dockerfile
│   └── src/
├── docker-compose.yaml          # Main orchestration file
├── .env.example                 # Environment variables template
├── README.md                    # Project documentation
└── AUTHORS.md                   # Team members
```

📄 Required Documentation

README.md

Your README must include:

Section	Content
Description	What problem does your app solve?
Technology Stack	Programming Language, Solutions used, etc.
Architecture	Diagram of your services
Installation	`git clone` + `docker compose up`
Environment	Table of all required `.env` variables

Section	Content
Demo	Screenshots or GIF

AUTHORS.md

```
# Team Members

- **Name 1** - Role/Contribution
- **Name 2** - Role/Contribution
- **Name 3** - Role/Contribution
- **Name 4** - Role/Contribution
```

📊 Evaluation Criteria

> 💡 ***Note:*** *With these criteria, you should be able to estimate your grade before the presentation.*

🎤 Presentation (10 points)

Criteria	Points	Details
Oral & Fluidity	2	English only, storytelling
Cooperation & Communication	3	Show how team worked together
Challenges Faced	2	Technical and soft skill challenges
Slide Quality	2	Visual, not text-heavy, includes diagrams
Live Demo	1	Working demonstration

💻 Project (10 points)

Criteria	Points	Details
Originality	2	Solves a real problem, creative
Network Isolation	1	Proven during presentation
Dockerfiles	2	Secure, optimized, well-structured

Criteria	Points	Details
Image Size	1	As small as possible
Data Persistence	1	Named volumes, survives restart
Hot Reload	1	Frontend updates without restart
Git Management	1	Branches, MRs, etc.
CI/CD + Security	1	Trivy + build + push working

🎁 Bonus Points

Bonus	Points	Description
Reverse Proxy	+0.5	Traefik, Nginx routing, etc.
Load Balancer	+0.5	Multiple replicas
Queue System	+0.5	Redis, RabbitMQ, etc.
GraphQL / gRPC	+0.5	Modern API technology
Multi-stage Builds	+0.5	Smaller final images
API Documentation	+0.5	OpenAPI / Swagger

📚 Useful Resources

- Docker Compose Documentation
- Dockerfile Best Practices
- Docker Networking
- Trivy Documentation

Good luck! 🍀